

MANUAL DE GITHUB DE NOVATO A AVANZADO

Guía exhaustiva para desarrolladores y administradores en Debian GNU/Linux con Bash, GitHub CLI y GitHub Actions

[CLI & Web Workflows](#)[SSH & GPG Signing](#)[GitHub Actions CI/CD](#)[Issues & Projects v2](#)[Security & Dependabot](#)[Packages & Releases](#)[API & Scripting](#)

Manual de GitHub: De Novato a Avanzado en Debian Linux

Plataforma: Debian GNU/Linux (11 Bullseye / 12 Bookworm y derivadas como Ubuntu / Linux Mint)

Shell: Bash / GNU Coreutils

Herramientas: Git 2.40+, GitHub CLI (`gh`), OpenSSH, GnuPG, Libsecret, Git LFS

Índice de Contenidos

1. [Parte I: Fundamentos y Configuración del Entorno en Debian](#)
 - 1.1 [Diferencias entre Git y GitHub](#)
 - 1.2 [Instalación de Git y GitHub CLI en Debian](#)
 - 1.3 [Configuración de Identidad y Finales de Línea](#)
 - 1.4 [Autenticación Segura: SSH Ed25519 y GitHub CLI](#)
 - 1.5 [Gestor de Credenciales en Linux](#)
 - 1.6 [Firma Criptográfica de Commits con GPG y SSH](#)
2. [Parte II: Flujo de Trabajo Esencial \(Nivel Novato\)](#)
 - 2.1 [Creación y Clonación de Repositorios](#)
 - 2.2 [El Ciclo de Tres Estados: Working Tree, Index y Commit](#)
 - 2.3 [Staging Selectivo y Convención de Commits](#)
 - 2.4 [Sincronización con el Repositorio Remoto](#)
 - 2.5 [Control de Archivos Ignorados](#)
3. [Parte III: Ramas, Fusiones y Estrategias Colaborativas \(Nivel Intermedio\)](#)
 - 3.1 [Ciclo de Vida de Ramas](#)
 - 3.2 [Los Tres Métodos de Fusión en GitHub](#)
 - 3.3 [Modelos de Flujo de Trabajo: GitHub Flow, Git Flow y Forking](#)
 - 3.4 [Herramientas de Respaldo: Stash, Cherry-Pick y Rebase Interactivo](#)
 - 3.5 [El Salvavidas: Recuperar Commits y Ramas con Reflog](#)
4. [Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo](#)
 - 4.1 [Tema 1: Prevención y Buenas Prácticas de Equipo](#)
 - 4.2 [Tema 2: Fusión Automática en Distintas Líneas](#)
 - 4.3 [Tema 3: Conflicto Directo de Fusión en Mismas Líneas](#)
 - 4.4 [Tema 4: Elección Total de Versión](#)
 - 4.5 [Tema 5: Cambios Locales sin Confirmar al hacer Pull](#)

- 4.6 [Tema 6: Push Rechazado por Desfase y Rebase Seguro](#)
- 4.7 [Tema 7: Resolución de Conflictos en Pull Requests](#)
- 4.8 [Tema 8: Conflicto de Modificación vs Eliminación](#)
- 4.9 [Tema 9: Archivos Binarios y Bloqueo con Git LFS](#)
- 5. [Parte V: Herramientas Modernas de Productividad Avanzada](#)
 - 5.1 [Git Worktrees: Múltiples Ramas en Paralelo sin Conmutar](#)
 - 5.2 [Depuración Binaria de Bugs con Git Bisect y Blame](#)
 - 5.3 [GitHub Codespaces y Contenedores de Desarrollo](#)
 - 5.4 [GitHub Copilot en la Terminal con GitHub CLI](#)
 - 5.5 [Git Hooks Locales y Automatización con Pre-commit](#)
- 6. [Parte VI: Gestión de Proyectos y Ecosistema GitHub](#)
 - 6.1 [GitHub Issues, Hitos y Etiquetas desde Terminal](#)
 - 6.2 [Pull Requests y Revisiones de Código desde el CLI](#)
 - 6.3 [GitHub Projects \(v2\): Tableros y Automatización](#)
 - 6.4 [GitHub Discussions y Wikis Locales](#)
- 7. [Parte VII: Automatización y CI/CD con GitHub Actions](#)
 - 7.1 [Estructura y Sintaxis de Workflows](#)
 - 7.2 [Pipelines para Debian/Linux: Tests, Linting y Matrices](#)
 - 7.3 [Secretos, Variables de Entorno y Caching](#)
 - 7.4 [Configuración de un Self-Hosted Runner en Debian](#)
- 8. [Parte VIII: Distribución, Paquetes y Publicación](#)
 - 8.1 [GitHub Releases: Tags Semánticos y Binarios](#) `.deb`
 - 8.2 [GitHub Packages: Contenedores en GHCR](#)
 - 8.3 [GitHub Pages: Despliegue de Sitios Estáticos](#)
- 9. [Parte IX: Seguridad, Gobernanza y Políticas de Repositorio](#)
 - 9.1 [Branch Protection Rules y Rulesets](#)
 - 9.2 [Dependabot, Secret Scanning y Push Protection](#)
 - 9.3 [Análisis Estático con CodeQL \(SAST\)](#)
 - 9.4 [Gobernanza con CODEOWNERS y Permisos](#)
- 10. [Parte X: Catálogo Maestro de Incidentes y Soluciones en GitHub](#)
 - 10.1 [Incidente 1: Fuga Accidental de Secretos o Tokens](#)
 - 10.2 [Incidente 2: Rechazo de Push por Archivo Mayor a 100 MB](#)
 - 10.3 [Incidente 3: Reversión Limpia de un Merge Roto en Producción](#)
 - 10.4 [Incidente 4: Rebase Accidental de una Rama Compartida](#)

- 10.5 [Incidente 5: Resurrección de una Rama Remota Borrada](#)
 - 10.6 [Incidente 6: Corrección Masiva de Autoría en Commits Antiguos](#)
 - 10.7 [Incidente 7: Ataques de Pwn Request en GitHub Actions](#)
 - 10.8 [Incidente 8: Bucle Infinito de Workflows en GitHub Actions](#)
 - 10.9 [Incidente 9: Repositorio Gigante y Poda de Objetos Huérfanos](#)
 - 10.10 [Incidente 10: Conflicto de Etiquetas o Tags Desincronizados](#)
-

Parte I: Fundamentos y Configuración del Entorno en Debian

1.1 Diferencias entre Git y GitHub

- **Git:** Sistema de control de versiones distribuido (DVCS) creado por Linus Torvalds. Se ejecuta localmente en tu máquina Debian, almacena la historia en `.git` y funciona completamente sin conexión a internet.
- **GitHub:** Plataforma en la nube construida alrededor de Git. Añade alojamiento centralizado, colaboración en equipo mediante *Pull Requests*, automatización con *GitHub Actions*, seguimiento con *Issues*, registro de paquetes y seguridad de código.

1.2 Instalación de Git y GitHub CLI en Debian

BASH

```
sudo apt update && sudo apt install -y curl wget git gnupg coreutils
```

¿Qué hace este comando?

Actualiza los índices de APT e instala las dependencias necesarias para gestionar repositorios, firmas y paquetes.

BASH

```
sudo mkdir -p -m 755 /etc/apt/keyrings
wget -qO- https://cli.github.com/packages/githubcli-archive-keyring.gpg | sudo tee
/etc/apt/keyrings/githubcli-archive-keyring.gpg > /dev/null
sudo chmod go+r /etc/apt/keyrings/githubcli-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/githubcli-archive-
keyring.gpg] https://cli.github.com/packages stable main" | sudo tee
/etc/apt/sources.list.d/github-cli.list > /dev/null
sudo apt update && sudo apt install -y gh
```

¿Qué hace este comando?

Registra la clave criptográfica oficial de GitHub, añade el repositorio oficial a Debian e instala la herramienta oficial de línea de comandos `gh`.

1.3 Configuración de Identidad y Finales de Línea

```
git config --global user.name "Tu Nombre Completo"
git config --global user.email "tu-correo@ejemplo.com"
git config --global core.autocrlf input
git config --global init.defaultBranch main
git config --global core.editor nano
```

¿Qué hace este comando?

Define el autor y correo para cada commit, fija los saltos de línea estrictamente en formato Unix LF (`\n`), configura `main` como rama inicial por omisión y define `nano` como editor de terminal.

1.4 Autenticación Segura: SSH Ed25519 y GitHub CLI

```
ssh-keygen -t ed25519 -C "tu-correo@ejemplo.com" -f ~/.ssh/id_ed25519
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

¿Qué hace este comando?

Crea una pareja de claves SSH usando la curva elíptica Ed25519 y carga la clave privada en el agente de memoria.

```
cat << 'EOF' >> ~/.ssh/config
Host github.com
    HostName ssh.github.com
    Port 443
    User git
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
EOF
chmod 600 ~/.ssh/config
```

¿Qué hace este comando?

Configura el acceso SSH a GitHub a través del puerto seguro **443**, previniendo bloqueos en redes que restringen el puerto 22.

```
gh auth login -p ssh -w  
ssh -T git@github.com
```

¿Qué hace este comando?

Autentica tu CLI con GitHub, asocia tu clave SSH pública automáticamente a tu perfil y comprueba la conexión.

1.5 Gestor de Credenciales en Linux

```
sudo apt install -y libsecret-1-0 libsecret-1-dev build-essential  
sudo make --directory=/usr/share/doc/git/contrib/credential/libsecret  
git config --global credential.helper /usr/share/doc/git/contrib/credential/libsecret/git-  
credential-libsecret
```

¿Qué hace este comando?

Compila e instala el ayudante de credenciales de GNOME/Linux para almacenar tokens PAT de manera cifrada en tu sesión.

1.6 Firma Criptográfica de Commits con GPG y SSH

```
git config --global gpg.format ssh  
git config --global user.signingkey ~/.ssh/id_ed25519.pub  
git config --global commit.gpgsign true  
git config --global tag.gpgsign true
```

¿Qué hace este comando?

Firma digitalmente cada commit y tag usando tu clave SSH pública para obtener la insignia **Verified** en GitHub.

Parte II: Flujo de Trabajo Esencial (Nivel Novato)

2.1 Creación y Clonación de Repositorios

```
mkdir mi-proyecto && cd mi-proyecto
git init
echo "# Mi Proyecto Debian" > README.md
git add README.md
git commit -m "docs: inicializar repositorio"
gh repo create mi-proyecto --public --source=. --remote=origin --push
```

¿Qué hace este comando?

Crea la carpeta, inicializa Git localmente, genera el primer commit y publica el repositorio en GitHub vinculando el remoto en un solo paso.

```
git clone git@github.com:usuario/mi-repositorio.git
```

¿Qué hace este comando?

Descarga una réplica completa de un repositorio existente y todo su árbol de commits.

2.2 El Ciclo de Tres Estados: Working Tree, Index y Commit

```
git status -s
git diff
git diff --staged
```

¿Qué hace este comando?

- `status -s`: Muestra el estado del árbol de trabajo resumido.
- `diff`: Muestra las diferencias no preparadas en disco.
- `diff --staged`: Muestra las diferencias preparadas en el staging.

2.3 Staging Selectivo y Convención de Commits

```
git add -p archivo.py
```

¿Qué hace este comando?

Abre el visor interactivo de bloques (hunks) para seleccionar manualmente qué líneas incluir en el commit.

```
git commit -m "feat(api): implementar endpoint de autenticación"
```

¿Qué hace este comando?

Crea un commit estructurado bajo la especificación **Conventional Commits** (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`).

2.4 Sincronización con el Repositorio Remoto

```
git fetch origin
git pull --rebase origin main
git push -u origin main
```

¿Qué hace este comando?

Descarga referencias remotas, incorpora los cambios situando tus commits ordenadamente arriba y los sube a GitHub.

2.5 Control de Archivos Ignorados

```
cat << 'EOF' > .gitignore
*.o
*.so
bin/
*.log
__pycache__/
.venv/
EOF
git rm --cached archivo_sensible.env
```

¿Qué hace este comando?

Define reglas de exclusión para artefactos y saca del control de versiones archivos rastreados por error sin borrarlos del disco.

Parte III: Ramas, Fusiones y Estrategias Colaborativas (Nivel Intermedio)

3.1 Ciclo de Vida de Ramas

```
git switch -c feature/nueva-funcionalidad
git push -u origin feature/nueva-funcionalidad
git switch main
git branch -d feature/nueva-funcionalidad
git push origin --delete feature/nueva-funcionalidad
```

¿Qué hace este comando?

Crea y conmuta a una rama, la publica en GitHub, regresa a `main`, borra la rama local ya integrada y la elimina en GitHub.

3.2 Los Tres Métodos de Fusión en GitHub

- **Merge Commit:** Fusión con commit conmemorativo que conserva todo el historial de ramas.
- **Squash and Merge:** Aplasta todos los commits en uno solo para mantener limpia la rama principal.
- **Rebase and Merge:** Aplica commits linealmente sin generar commit de unión.

3.3 Modelos de Flujo de Trabajo: GitHub Flow, Git Flow y Forking

```
gh repo fork organizacion/proyecto-upstream --clone
cd proyecto-upstream
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

¿Qué hace este comando?

Crea un fork personal de un proyecto Open Source, lo clona localmente y mantiene sincronizada tu copia con el repositorio original.

3.4 Herramientas de Respaldo: Stash, Cherry-Pick y Rebase Interactivo

```
git stash save "Trabajo preliminar sin terminar"
git stash pop
git cherry-pick 3a5b7c8
git rebase -i HEAD~3
```

¿Qué hace este comando?

Guarda cambios en el búfer temporal, los restaura, copia un commit específico de otra rama o reescribe interactivamente los últimos 3 commits.

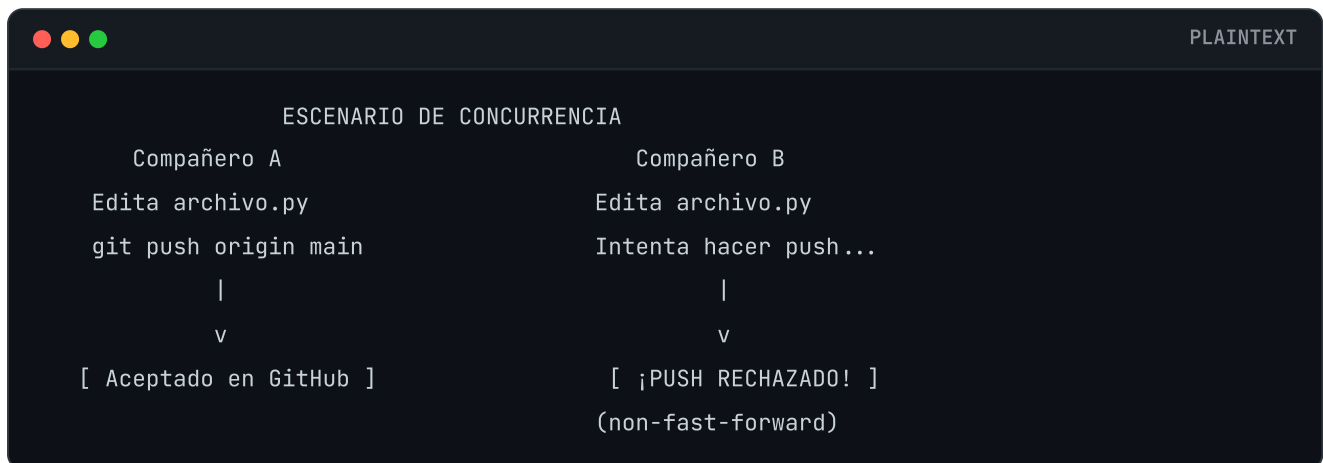
3.5 El Salvavidas: Recuperar Commits y Ramas con Reflog

```
git reflog
git reset --hard HEAD@{1}
```

¿Qué hace este comando?

Inspecciona el historial de todos los movimientos de `HEAD` en tu máquina y revierte cualquier pérdida accidental a su estado previo.

Parte IV: Soluciones por Temas a la Edición Concurrente del Mismo Archivo



4.1 Tema 1: Prevención y Buenas Prácticas de Equipo

1. **Ramas por funcionalidad:** Prohibir commits directos sobre `main`.
2. **Modularización:** Fragmentar archivos masivos en módulos pequeños.
3. **Draft PRs:** Abrir Pull Requests en borrador para visibilizar qué archivos están siendo alterados.
4. **Sincronizaciones frecuentes:** Ejecutar `git pull --rebase origin main` al iniciar y finalizar la jornada.

4.2 Tema 2: Fusión Automática en Distintas Líneas

Si las modificaciones ocurrieron en secciones separadas del archivo:

```
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Descarga los cambios del compañero y coloca tus commits arriba. Al no haber solapamiento de líneas, Git resuelve la fusión automáticamente.

4.3 Tema 3: Conflicto Directo de Fusión en Mismas Líneas

Cuando ambos tocaron las mismas líneas, Git añade marcas de conflicto:

```
PYTHON

<<<<<<< HEAD (Tu compañero en GitHub)
puerto_servicio = 8080
=====
puerto_servicio = 9090
>>>>>> feat: mi cambio local
```

```
BASH

git status
nano src/archivo.py
git add src/archivo.py
git rebase --continue
git push origin main
```

¿Qué hace este comando?

Identifica los archivos en conflicto, te permite editar y borrar los marcadores `<<<<<<<`, marca el archivo como resuelto con `git add`, continúa el rebase y sube los cambios.

```
BASH

git rebase --abort
```

¿Qué hace este comando?

Cancela inmediatamente el proceso de rebase o fusión y devuelve el repositorio a su estado previo al conflicto.

4.4 Tema 4: Elección Total de Versión

```
# Conservar mi versión completa y descartar la del compañero:
git checkout --ours src/archivo.py
git add src/archivo.py && git commit -m "resolve: mantener versión local"

# Aceptar la versión remota del compañero completa:
git checkout --theirs src/archivo.py
git add src/archivo.py && git commit -m "resolve: aceptar versión remota"
```

¿Qué hace este comando?

Resuelve el conflicto escogiendo en su totalidad uno de los dos lados sin edición línea por línea.

4.5 Tema 5: Cambios Locales sin Confirmar al hacer Pull

```
git stash save "Cambios locales en curso"
git pull --rebase origin main
git stash pop
```

¿Qué hace este comando?

Guarda tus modificaciones no confirmadas en la pila temporal, actualiza el repositorio con los cambios remotos y reaplica tus cambios locales.

4.6 Tema 6: Push Rechazado por Desfase y Rebase Seguro

```
# 1. NUNCA fuerces con push --force (borrarías los commits de tu compañero)
git pull --rebase origin main
git push origin main
```

¿Qué hace este comando?

Reubica tus commits locales por encima de los que tu compañero acaba de subir a GitHub y publica la rama sin sobrescribir nada.

4.7 Tema 7: Resolución de Conflictos en Pull Requests

```
gh pr checkout 25
git fetch origin main
git merge origin/main
nano src/archivo.py
git add src/archivo.py
git commit -m "merge: reconciliar conflictos con main"
git push origin HEAD
```

¿Qué hace este comando?

Descarga la rama del PR localmente con `gh`, incorpora `main`, resuelve conflictos y actualiza el PR en GitHub.

4.8 Tema 8: Conflicto de Modificación vs Eliminación

```
# Para conservar el archivo frente a la eliminación:
git add src/archivo.py && git commit -m "resolve: conservar archivo modificado"

# Para confirmar la eliminación:
git rm src/archivo.py && git commit -m "resolve: confirmar borrado de archivo"
```

¿Qué hace este comando?

Resuelve situaciones donde un usuario modificó el archivo y otro lo borró.

4.9 Tema 9: Archivos Binarios y Bloqueo con Git LFS

```
sudo apt install -y git-lfs && git lfs install
git lfs track "*.psd" --lockable
git lfs lock assets/disenio.psd
git lfs locks
git lfs unlock assets/disenio.psd
```

¿Qué hace este comando?

Activa el bloqueo exclusivo de archivos en GitHub para impedir que dos personas editen un archivo binario a la vez.

Parte V: Herramientas Modernas de Productividad Avanzada

5.1 Git Worktrees: Múltiples Ramas en Paralelo sin Conmutar

Git Worktrees permite tener múltiples ramas extraídas simultáneamente en carpetas independientes en disco sin usar `git stash` ni conmutar ramas en el directorio principal:

```
BASH

# Crear un worktree paralelo para una corrección urgente en ../hotfix-login
git worktree add ../hotfix-login hotfix/login-bug
cd ../hotfix-login

# Realizar cambios y commitear en paralelo
git commit -am "fix: resolver problema de sesión"
git push origin hotfix/login-bug

# Regresar al proyecto principal y eliminar el worktree al terminar
cd ../mi-proyecto
git worktree list
git worktree remove ../hotfix-login
```

¿Qué hace este comando?

Asocia una rama diferente a una carpeta física paralela sin tocar tu directorio de trabajo actual.

5.2 Depuración Binaria de Bugs con Git Bisect y Blame

Cuando descubres un error en producción y no sabes qué commit lo introdujo:

```
# Iniciar búsqueda binaria
git bisect start
git bisect bad           # La versión actual contiene el error
git bisect good v1.0.0   # El tag v1.0.0 funcionaba correctamente

# Git conmutará automáticamente a commits intermedios para que pruebes
# Tras probar cada commit, ejecutas:
git bisect good          # 0: git bisect bad

# Al terminar, Git te indicará el commit exacto causante. Finaliza con:
git bisect reset
```

¿Qué hace este comando?

Ejecuta una búsqueda binaria algorítmica sobre el historial de commits para aislar el commit culpable en segundos.

```
git blame -L 40,60 src/servidor.py
```

¿Qué hace este comando?

Muestra línea por línea quién fue el autor del último cambio entre las líneas 40 y 60, junto con el hash del commit y la fecha.

5.3 GitHub Codespaces y Contenedores de Desarrollo

Configura entornos de desarrollo inmediatos y reproducibles en la nube de GitHub mediante `.devcontainer/devcontainer.json`:

JSON

```
{
  "name": "Debian Linux Dev Environment",
  "image": "mcr.microsoft.com/devcontainers/base:debian",
  "features": {
    "ghcr.io/devcontainers/features/github-cli:1": {}
  },
  "customizations": {
    "vscode": {
      "extensions": ["ms-azuretools.vscode-docker", "eamodio.gitlens"]
    }
  }
}
```

BASH

```
# Iniciar o conectar a un Codespace desde Debian con GitHub CLI
gh codespace create --repo usuario/mi-proyecto --branch main
gh codespace list
gh codespace code -c nombre-del-codespace
```

¿Qué hace este comando?

Despliega una máquina virtual en la infraestructura de GitHub configurada con tus herramientas y la abre en tu terminal o editor.

5.4 GitHub Copilot en la Terminal con GitHub CLI

BASH

```
# Instalar la extensión oficial de Copilot para gh
gh extension install github/gh-copilot

# Solicitar sugerencias de comandos de terminal
gh copilot suggest "listar procesos que más memoria consumen en Debian"

# Pedir explicación de un comando complejo
gh copilot explain "iptables -t nat -A PREROUTING -p tcp --dport 80 -j REDIRECT --to-port 8080"
```

¿Qué hace este comando?

Integra inteligencia artificial generativa dentro de Bash para generar comandos o explicar sintaxis complejas de Linux.

5.5 Git Hooks Locales y Automatización con Pre-commit

BASH

```
sudo apt install -y python3-pip
pip install pre-commit --break-system-packages

cat << 'EOF' > .pre-commit-config.yaml
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: check-added-large-files
      args: ['--maxkb=5000']
    - id: detect-private-key
    - id: trailing-whitespace
    - id: end-of-file-fixer
EOF

pre-commit install
```

¿Qué hace este comando?

Instala ganchos automáticos que interceptan cada `git commit`, bloqueando la confirmación si detecta claves privadas o archivos mayores a 5 MB.

Parte VI: Gestión de Proyectos y Ecosistema GitHub

6.1 GitHub Issues, Hitos y Etiquetas desde Terminal

```
gh issue create --title "Fallo en parseo de JSON" --body "El sistema falla con cadenas vacías." --label "bug,debian"
gh issue list --assignee "@me"
gh issue close 42 --reason "completed"
```

¿Qué hace este comando?

Crea, audita y cierra incidencias formales vinculadas al repositorio desde la consola de Debian.

6.2 Pull Requests y Revisiones de Código desde el CLI

```
gh pr create --title "feat: integración con systemd" --body "Automatiza el arranque del servicio." --draft
gh pr checkout 15
gh pr review 15 --approve -b "Validado en Debian 12 sin fallos."
```

¿Qué hace este comando?

Crea un Pull Request en borrador, descarga la rama de un compañero a tu equipo y emite una aprobación oficial.

6.3 GitHub Projects (v2): Tableros y Automatización

```
gh project list
gh project item-add 3 --owner "mi-organizacion" --url
"https://github.com/usuario/repo/issues/42"
```

¿Qué hace este comando?

Gestiona tableros Kanban interactivos y asocia tareas automáticamente.

6.4 GitHub Discussions y Wikis Locales

BASH

```
git clone git@github.com:usuario/mi-proyecto-debian.wiki.git
cd mi-proyecto-debian.wiki
echo "## Especificaciones de Arquitectura" >> Home.md
git commit -am "docs: expandir wiki técnica" && git push origin master
```

¿Qué hace este comando?

Clona la documentación Wiki de GitHub como un repositorio Git local, añade contenido y la publica.

Parte VII: Automatización y CI/CD con GitHub Actions

7.1 Estructura y Sintaxis de Workflows

Definidos en `.github/workflows/*.yaml` para compilar, auditar y desplegar código ante eventos de Git.

7.2 Pipelines para Debian/Linux: Tests, Linting y Matrices

Crea `.github/workflows/debian-ci.yaml` :

```

name: Debian Linux CI Pipeline

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ["3.10", "3.11", "3.12"]

    steps:
      - name: Descargar repositorio
        uses: actions/checkout@v4

      - name: Configurar entorno Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }
          cache: 'pip'

      - name: Instalar paquetes del sistema y dependencias
        run: |
          sudo apt update && sudo apt install -y build-essential libpq-dev
          pip install --upgrade pip
          if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
          pip install flake8 pytest

      - name: Ejecutar análisis estático (Linting)
        run: flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics

      - name: Ejecutar pruebas unitarias
        run: pytest --junitxml=reports/test-results-${ matrix.python-version }.xml

      - name: Subir reporte de pruebas como artefacto
        uses: actions/upload-artifact@v4
        if: always()

```

```
with:
  name: test-results-${{ matrix.python-version }}
  path: reports/
```

7.3 Secretos, Variables de Entorno y Caching

```
- name: Despliegue seguro a servidor Debian
  env:
    SSH_KEY: ${ secrets.PROD_SERVER_SSH_KEY }
    HOST: ${ vars.PROD_HOST_IP }
  run: |
    echo "$SSH_KEY" > key.pem && chmod 600 key.pem
    ssh -i key.pem -o StrictHostKeyChecking=no admin@$HOST "systemctl restart mi-servicio"
```

7.4 Configuración de un Self-Hosted Runner en Debian

```
sudo adduser --disabled-password --gecos "" actions-runner
sudo usermod -aG docker actions-runner
sudo su - actions-runner
mkdir actions-runner && cd actions-runner
curl -o actions-runner-linux-x64-2.316.0.tar.gz -L
https://github.com/actions/runner/releases/download/v2.316.0/actions-runner-linux-x64-
2.316.0.tar.gz
tar xzf ./actions-runner-linux-x64-2.316.0.tar.gz
./config.sh --url https://github.com/usuario/repo --token TU_TOKEN_AQUI
sudo ./svc.sh install actions-runner
sudo ./svc.sh start
sudo ./svc.sh status
```

¿Qué hace este comando?

Convierte tu servidor Debian en un ejecutor privado de GitHub Actions registrado como servicio Systemd.

Parte VIII: Distribución, Paquetes y Publicación

8.1 GitHub Releases: Tags Semánticos y Binarios .deb

```
git tag -a v1.0.0 -m "release: versión 1.0.0 estable"
git push origin v1.0.0
gh release create v1.0.0 ./dist/mi-app_1.0.0_amd64.deb --title "Versión 1.0.0 para Debian"
--generate-notes
```

¿Qué hace este comando?

Publica formalmente una nueva versión de software en GitHub y adjunta el paquete instalable para Debian.

8.2 GitHub Packages: Contenedores en GHCR

```
echo $CR_PAT | docker login ghcr.io -u TU_USUARIO --password-stdin
docker build -t ghcr.io/tu-usuario/mi-app:1.0.0 .
docker push ghcr.io/tu-usuario/mi-app:1.0.0
```

¿Qué hace este comando?

Publica imágenes Docker en el registro de contenedores de GitHub.

8.3 GitHub Pages: Despliegue de Sitios Estáticos

Permite alojar portales estáticos y sitios de documentación con despliegue automático mediante GitHub Actions.

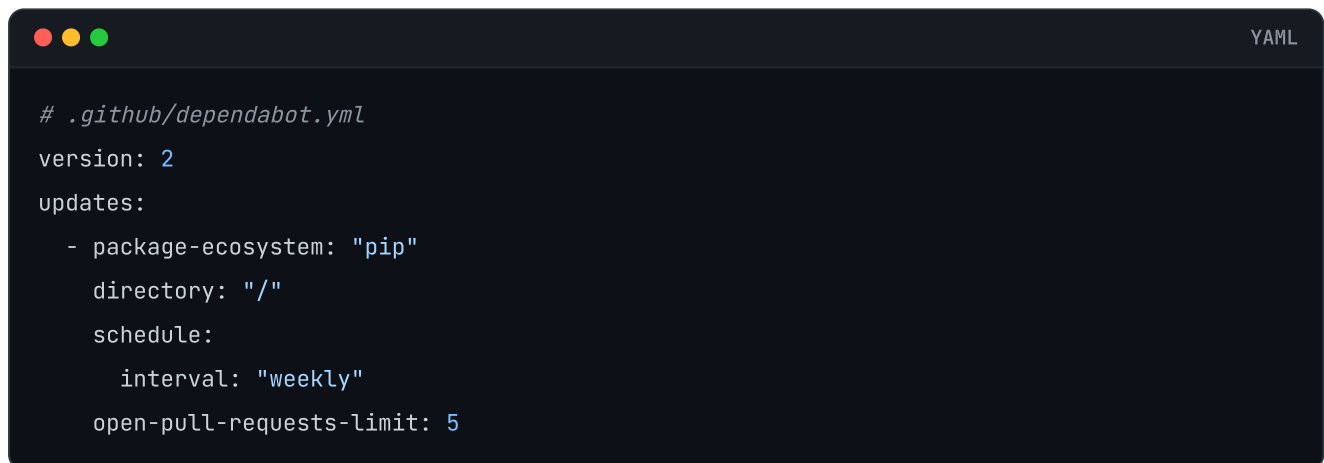
Parte IX: Seguridad, Gobernanza y Políticas de Repositorio

9.1 Branch Protection Rules y Rulesets

Desde **Settings** → **Rules** → **Rulesets**:

- Prohibir commits directos a `main`.
- Requerir revisiones aprobadas obligatorias.
- Exigir que los pipelines de pruebas pasen satisfactoriamente.
- Bloquear force-pushes (`git push --force`).

9.2 Dependabot, Secret Scanning y Push Protection

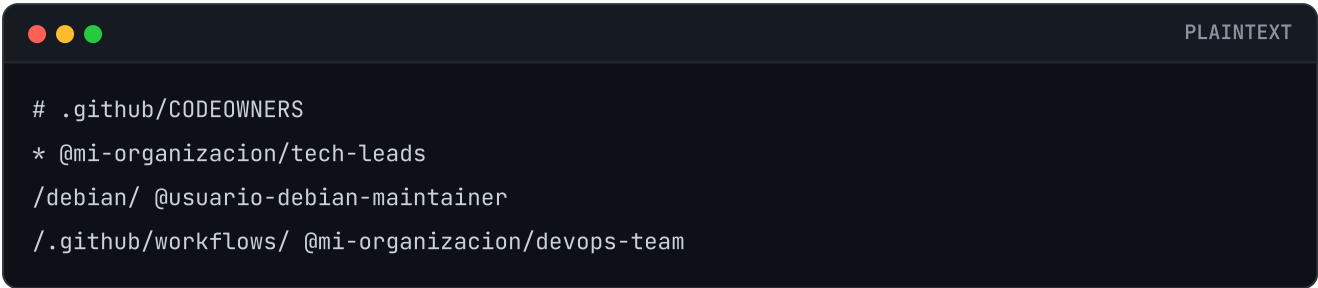
A screenshot of a code editor window with a dark theme. The title bar shows three colored window control buttons (red, yellow, green) on the left and the text 'YAML' on the right. The code is a GitHub Actions workflow for Dependabot, written in YAML. It starts with a comment line, followed by 'version: 2', 'updates:', and a list of update configurations for the 'pip' package ecosystem. The configuration includes a directory of '/', a weekly schedule, and an open-pull-requests-limit of 5.

```
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "pip"
    directory: "/"
    schedule:
      interval: "weekly"
    open-pull-requests-limit: 5
```

9.3 Análisis Estático con CodeQL (SAST)

Escaneo automatizado de vulnerabilidades de código fuente integrado directamente en cada Pull Request.

9.4 Gobernanza con CODEOWNERS y Permisos



```
# .github/CODEOWNERS
* @mi-organizacion/tech-leads
/debian/ @usuario-debian-maintainer
/.github/workflows/ @mi-organizacion/devops-team
```

A terminal window with a dark background and a title bar. The title bar has three colored circles (red, yellow, green) on the left and the word "PLAINTEXT" on the right. The terminal displays the content of a .github/CODEOWNERS file.

Parte X: Catálogo Maestro de Incidentes y Soluciones en GitHub

En entornos reales de producción, ocurren incidentes críticos que van más allá del flujo habitual. Esta sección cataloga las 10 situaciones más delicadas y su solución definitiva:

10.1 Incidente 1: Fuga Accidental de Secretos o Tokens

Escenario: Hiciste commit y push a GitHub de un archivo `.env` o script con una clave de API, contraseña o clave privada SSH.

CAUTION

REGLA DE ORO: Una vez que un secreto llega a GitHub, debes considerarlo inmediatamente comprometido. Aunque lo borres en un commit posterior, permanece accesible en el historial y en los forks.

Protocolo de remediación inmediata:

1. **Revocar y rotar el secreto:** Acude al proveedor (AWS, Stripe, OpenAI, etc.) y revoca inmediatamente la clave comprometida antes de tocar Git.
2. **Purgar el archivo del historial completo:** Instala `git-filter-repo` en Debian:

```
sudo apt install -y git-filter-repo
git filter-repo --path archivo_sensible.env --invert-paths --force
```

¿Qué hace este comando?

Reescribe cada commit del historial eliminando cualquier rastro del archivo sensible de la base de datos de Git.

3. **Forzar la sincronización segura:**

```
git push origin --force --all
git push origin --force --tags
```

10.2 Incidente 2: Rechazo de Push por Archivo Mayor a 100 MB

Escenario: GitHub rechaza el push con el error:

```
remote: error: File dataset.zip is 120.00 MB; this exceeds GitHub's file size limit of 100.00 MB .
```

```
BASH

# Caso A: El archivo está en el commit más reciente (aún no pusheado con éxito)
git reset --soft HEAD~1
git rm --cached dataset.zip
echo "dataset.zip" >> .gitignore
git commit -m "docs: reconstruir commit sin archivo pesado"
git push origin main
```

```
BASH

# Caso B: El archivo quedó atrapado en commits anteriores del historial local
git filter-repo --strip-blobs-bigger-than 100M --force
git push origin main
```

¿Qué hace este comando?

Elimina automáticamente cualquier objeto mayor a 100 MB de todo el historial local para que GitHub acepte el push.

10.3 Incidente 3: Reversión Limpia de un Merge Roto en Producción

Escenario: Se fusionó un Pull Request a `main` y provocó una caída en producción. No puedes hacer `git reset` porque alterarías el historial de todos los colaboradores.

```
BASH

# Localizar el hash del commit de merge
git log --oneline -n 5

# Revertir el commit de merge indicando la rama padre principal (-m 1)
git revert -m 1 HASH_DEL_MERGE -m "revert: revertir merge defectuoso de feature/login"
git push origin main
```

¿Qué hace este comando?

Crea un nuevo commit que anula exactamente las modificaciones introducidas por el merge,

preservando la continuidad del árbol y solucionando la emergencia al instante.

10.4 Incidente 4: Rebase Accidental de una Rama Compartida

Escenario: Un compañero ejecutó `git rebase` sobre una rama que varios miembros estaban utilizando y forzó el push, desalineando los clones de todos los demás.

Solución para los demás miembros del equipo:

```
BASH

# 1. Traer las referencias del servidor sin mezclar
git fetch origin

# 2. Situar la rama local sobre la rama remota reescrita sin duplicar commits
git switch rama-afectada
git rebase --onto origin/rama-afectada @{upstream}
```

¿Qué hace este comando?

Identifica los commits propios que aún no estaban en el rebase del compañero y los traslada limpiamente sobre la nueva base, evitando commits duplicados.

10.5 Incidente 5: Resurrección de una Rama Remota Borrada

Escenario: Alguien eliminó una rama importante en GitHub mediante la interfaz web o con `git push origin --delete rama-vital`.

```
BASH

# 1. Si algún miembro tenía la rama localmente:
git switch rama-vital
git push -u origin rama-vital

# 2. Si la rama fue borrada incluso de tu máquina local, busca el último commit en reflog:
git reflog | grep "rama-vital"
# Identifica el hash, por ejemplo 9a8b7c6

# 3. Resucitar la rama a partir del commit localizado:
git switch -c rama-vital 9a8b7c6
git push -u origin rama-vital
```

¿Qué hace este comando?

Reconstruye la rama exactamente en el estado del último commit antes de ser borrada y la vuelve a publicar en GitHub.

10.6 Incidente 6: Corrección Masiva de Autoría en Commits Antiguos

Escenario: Realizaste decenas de commits con un correo personal o erróneo (`usuario@localhost`) y GitHub no los vincula a tus estadísticas de contribución.

```
git filter-repo --email-callback '
return email.replace(b"correo_viejo@ejemplo.com", b"correo_nuevo@ejemplo.com")
' --force
git push origin --force --all
```

¿Qué hace este comando?

Reescribe todos los commits históricos sustituyendo el correo electrónico antiguo por el verificado en tu cuenta de GitHub.

10.7 Incidente 7: Ataques de Pwn Request en GitHub Actions

Escenario: Un atacante envía un Pull Request desde un fork malicioso intentando ejecutar código que lea tus secretos de producción.

Medidas de seguridad y configuración obligatoria:

1. En GitHub acude a **Settings** → **Actions** → **General** → **Fork pull request workflows**.
2. Selecciona **Require approval for all outside collaborators**.
3. **NUNCA utilices el evento** `pull_request_target` para ejecutar código extraído del checkout del PR si ese workflow tiene acceso a secretos sensibles. Utiliza siempre el evento estándar `pull_request` (que se ejecuta sin acceso a secretos de entorno).

10.8 Incidente 8: Bucle Infinito de Workflows en GitHub Actions

Escenario: Tienes un workflow de formateo o linting que hace `git commit` y `git push` automáticamente, provocando que el push dispare de nuevo el workflow indefinidamente consumiendo tus minutos de CI.

Solución: Añadir `[skip ci]` al mensaje del commit automatizado:

```
YAML
- name: Guardar formateo automático
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add .
    git diff-index --quiet HEAD || git commit -m "style: auto-formateo de código [skip ci]"
    git push
```

¿Qué hace este comando?

La etiqueta `[skip ci]` en el mensaje le indica al motor de GitHub Actions que ignore ese push específico y no vuelva a desencadenar el pipeline.

10.9 Incidente 9: Repositorio Gigante y Poda de Objetos Huérfanos

Escenario: Tu repositorio tarda demasiado en clonarse porque acumula años de referencias y objetos huérfanos en la base de datos de Git.

```
BASH
# 1. Comprobar el tamaño real de los objetos en disco
git count-objects -vH

# 2. Purgar referencias expiradas del reflog
git reflog expire --expire=now --all

# 3. Compactar agresivamente la base de datos de objetos
git gc --prune=now --aggressive
```

¿Qué hace este comando?

Empaqueta y optimiza los árboles de objetos de `.git`, eliminando archivos colgantes y reduciendo significativamente el peso del repositorio.

10.10 Incidente 10: Conflicto de Etiquetas o Tags Desincronizados

Escenario: Alguien movió o recreó una etiqueta de versión en GitHub y al hacer `git pull` recibes:

```
fatal: tag 'v1.0.0' already exists .
```

```
# 1. Purgar todas las etiquetas locales desincronizadas
git tag -d $(git tag -l)

# 2. Descargar las etiquetas oficiales y actualizadas directamente de GitHub
git fetch --tags --prune origin
```

¿Qué hace este comando?

Limpiar la caché local de etiquetas y fuerza la descarga limpia de los tags tal como existen en GitHub.